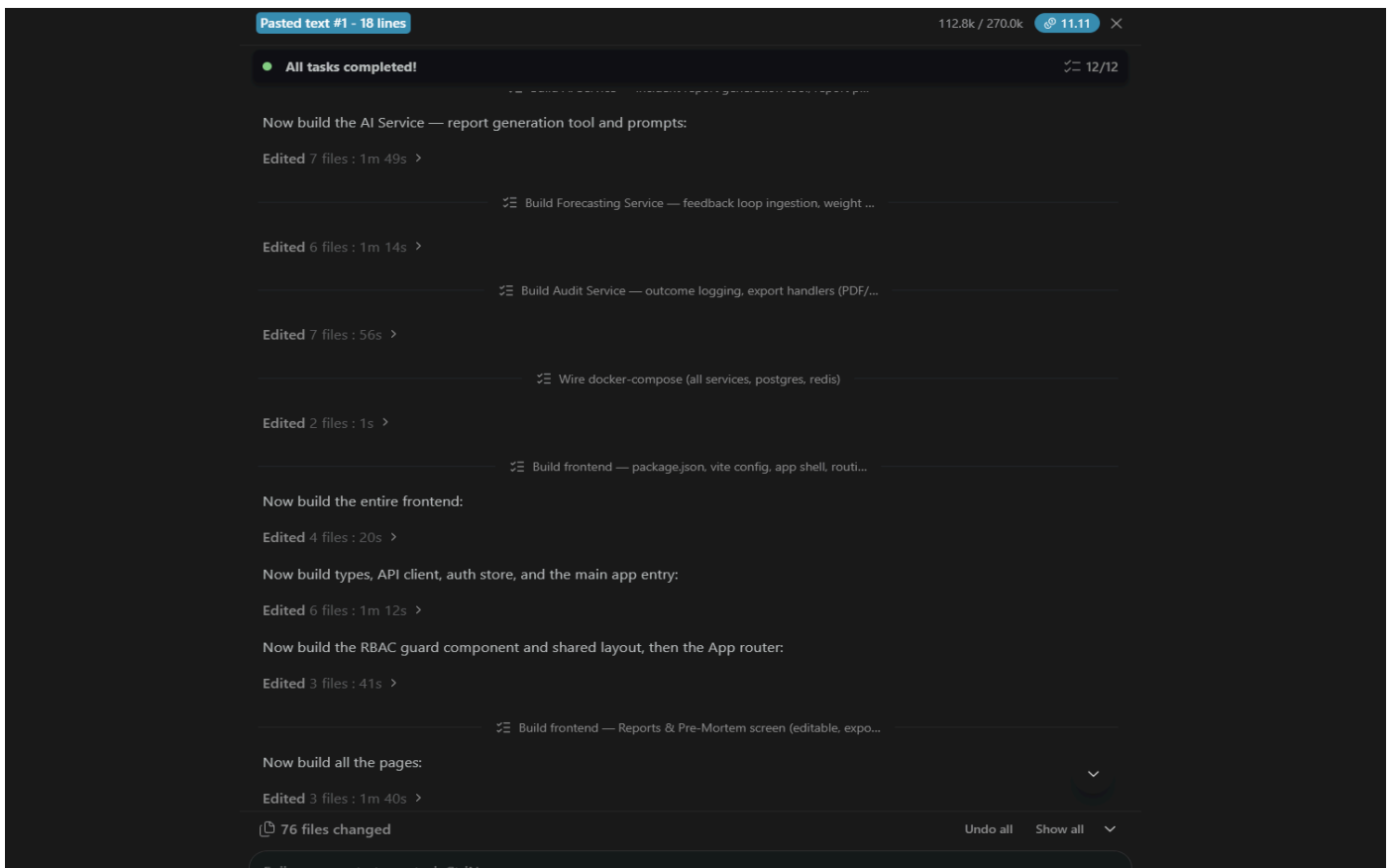
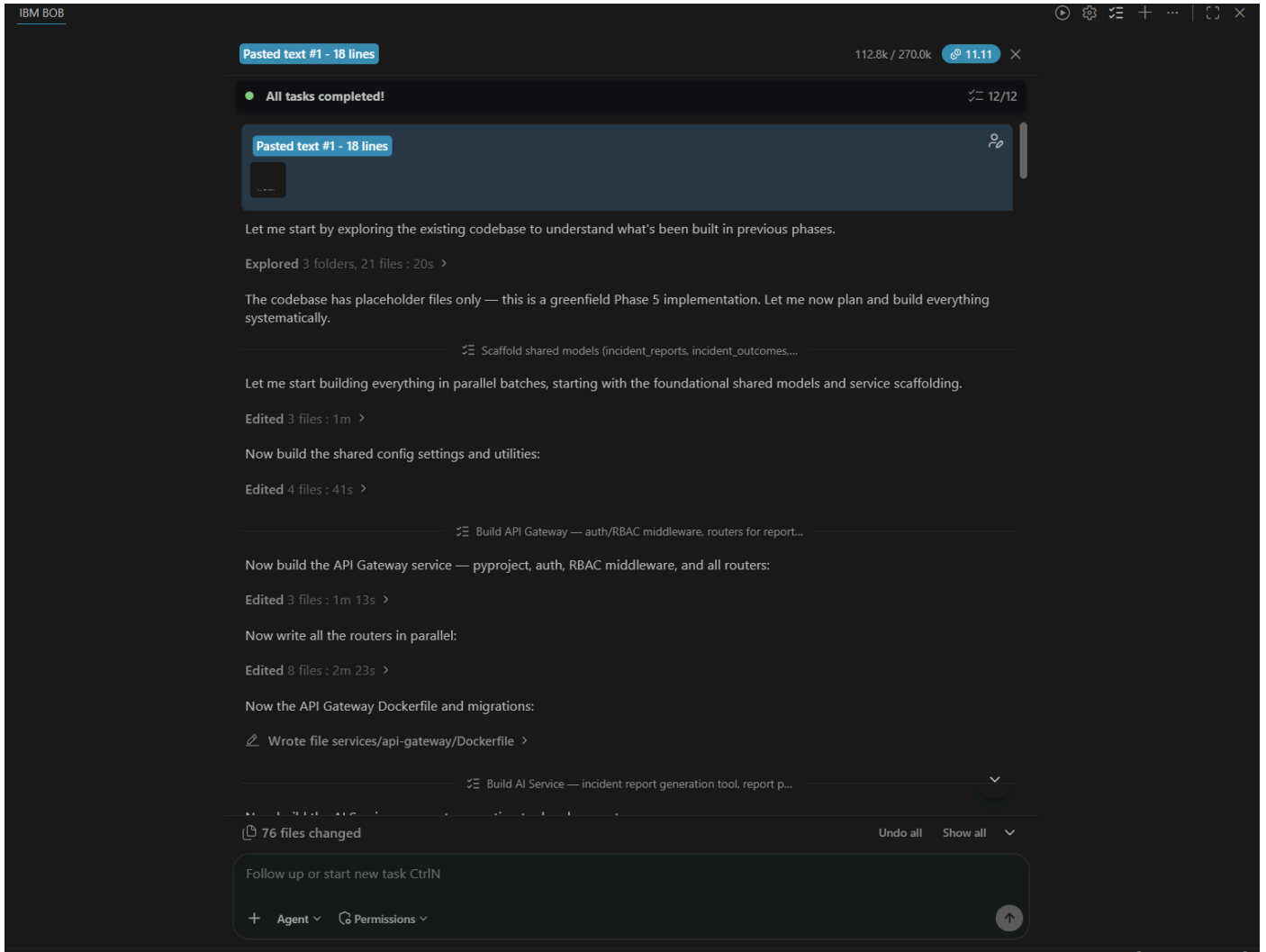




Now build all the pages:

Edited 3 files : 1m 40s >

Build frontend — Deployment Comparison screen

Edited 5 files : 53s >

Now the frontend Dockerfile, then pyproject files for remaining services, and the Makefile:

Edited 2 files : 2s >

Write pyproject.toml files for each service

Edited 3 files : 2s >

Now write the `.env.example` and the Makefile:

Edited 2 files : 995ms >

Now write the RBAC smoke test script and API docs:

Wrote file docs/scripts/rbac_smoke_test.py >

Now write the API docs and the Phase 5 architecture doc:

Edited 2 files : 47s >

All 12 items complete. Let me do a final structural validation pass to make sure everything connects cleanly:

Ran `Get-ChildItem -Recurse -File -Path . -Exclude "*.gitkeep" | Where-Object { $_.FullName -notmatch "\\g..." }` >

Everything is in order. Let me verify the key cross-service wiring is consistent by checking the forecasting weights API path referenced from the frontend against the actual forecasting service router: >

76 files changed

Undo all Show all >

Pasted text #1 - 18 lines

112.8k / 270.0k @ 11.11 X

All tasks completed!

12/12

All 12 items complete. Let me do a final structural validation pass to make sure everything connects cleanly:

Ran `Get-ChildItem -Recurse -File -Path . -Exclude "*.gitkeep" | Where-Object { $_.FullName -notmatch "\\g..." }` >

Everything is in order. Let me verify the key cross-service wiring is consistent by checking the forecasting weights API path referenced from the frontend against the actual forecasting service router:

Searched files for "feedback/weights/feedback" >

The forecasting weights are served by the forecasting service on port 8002, not via the API gateway. The frontend proxies everything through `/api` → gateway on 8000. The weights endpoint needs to be proxied through the gateway. Let me add a proxy route for the forecasting feedback endpoint in the gateway:

Edited 3 files : 17s >

Now verify the `shared/__init__.py` properly exports the module:

Read file `shared/__init__.py` >

Wrote file `shared/__init__.py` >

Now do a final cross-reference check — make sure the `pydantic-settings` package is listed in the shared config (it imports `BaseSettings` from it):

Applied diff to `shared/pyproject.toml` >

Phase 5 is complete. Here's a full summary of everything shipped:

Phase 5 — What was built

Shared Layer (`shared/`)

- `shared/models/__init__.py` — 5 SQLAlchemy ORM models: `User`, `IncidentReport`, `IncidentOutcome`, `ForecastWeight`, `DeploymentSnapshot`, plus `Role`, `Permission`, `ROLE_PERMISSIONS`, and `has_permission()` — the canonical RBAC source-of-truth
- `shared/config/settings.py` — `pydantic-settings` base config
- `shared/utils/db.py` — `get_session()` FastAPI dependency, `init_db()`

76 files changed

Undo all Show all >

Pasted text #1 - 18 lines

112.8k / 270.0k 11.11 X

All tasks completed!

✓ 12/12

API Gateway (services/api-gateway/)

- app/auth/__init__.py — JWT mint/verify, bcrypt password helpers, require_permission(perm) FastAPI dependency — the RBAC enforcer used on every endpoint
- app/routers/auth.py — /auth/token, /auth/register, /auth/me
- app/routers/reports.py — POST /reports/generate (calls AI service, persists draft), CRUD + GET /reports/{id}/export/{pdf|markdown}
- app/routers/outcomes.py — POST /outcomes (persists outcome, non-blocking call to forecasting service weight update)
- app/routers/deployments.py — GET /deployments/compare structured config diff
- app/routers/users.py — Admin-only user management
- app/routers/feedback.py — Proxies /feedback/weights to forecasting service

AI Service (services/ai-service/)

- app/prompts/report_prompts.py — Versioned system + user prompt templates for structured JSON output (Summary, Timeline, Affected Services, Impact, Root Cause, Resolution, Preventive Actions)
- app/reports/generator.py — generate_incident_report() calls GPT-4o with response_format: json_object; automatic dev stub when no API key is configured so the service starts without credentials

Forecasting Service (services/forecasting-service/)

- app/loop/feedback.py — Online learning weight update: TP→+lr, FP→-lr, FN→-0.5-lr; maintains running precision/recall per indicator
- app/loop/__init__.py — POST /feedback/ingest, GET /feedback/weights

Audit Service (services/audit-service/)

- app/exporters/__init__.py — report_to_markdown() and report_to_pdf_bytes() (WeasyPrint + Jinja2, falls back to HTML if WeasyPrint unavailable)
- app/templates/report.html.j2 — Print-ready HTML template with severity badges and markdown table rendering
- app/handlers/__init__.py — POST /export/pdf, POST /export/markdown

Frontend (frontend/src/)

76 files changed

Undo all Show all

Follow up or start new task Ctrl+N

+ Agent Permissions



All tasks completed!

12/12

- pages/LoginPage.tsx — JWT login form
- pages/ReportsPage.tsx — Reports list + **Generate Report** modal (incident context → AI draft in < 2 min)
- pages/ReportDetailPage.tsx — Fully editable section-by-section with Markdown preview, PDF/MD export buttons, **Log Outcome** modal that fires the feedback loop
- pages/DeploymentsPage.tsx — Side-by-side deployment picker → colour-coded config diff table
- pages/WeightsPage.tsx — Live-refreshing indicator weight table with precision/recall bars; auto-polls every 10s
- pages/UsersPage.tsx — Admin-only inline role editor
- components/shared/RoleGuard.tsx — Route guard → /login or /403
- components/shared/Layout.tsx — RBAC-aware sidebar that hides admin-only links from non-admins

Infrastructure

- docker-compose.yml — All 7 services (postgres, redis, api-gateway, ai-service, forecasting-service, audit-service, frontend) with health checks and hot-reload volumes
- docs/scripts/rbac_smoke_test.py — CLI script that registers one user per role and asserts correct 200/403 responses across every endpoint; run via `make rbac-smoke`
- Makefile — `make up`, `make test`, `make rbac-smoke`, `make frontend-dev`, etc.

error in tsconfig.json

Let me read the externally modified files first.

Explored 4 files: 4ms

The issue is `"moduleResolution": "bundler"` combined with `"allowImportingTsExtensions": true` and `"noEmit": true`. This combination is valid for **type-checking only** (Vite handles the build), but the `build` script in `package.json` runs `tsc && vite build` — `tsc` with `noEmit: true` won't emit anything and will fail when it encounters `.tsx` extension imports because `allowImportingTsExtensions` requires `noEmit` to be `true` which it is, but `moduleResolution: "bundler"` is only available in TypeScript 5.0+. That's fine here.

The actual problem: `"moduleResolution": "bundler"` is incompatible with `"module": "ESNext"` when running `tsc` for **type-check in strict mode in TS 5.4** — specifically, Vite projects should use a separate `tsconfig.node.json` for `vite.config.ts` (which uses

76 files changed

Undo all Show all

All tasks completed!

12/12

error in tsconfig.json

Let me read the externally modified files first.

Explored 4 files: 4ms

The issue is `"moduleResolution": "bundler"` combined with `"allowImportingTsExtensions": true` and `"noEmit": true`. This combination is valid for **type-checking only** (Vite handles the build), but the `build` script in `package.json` runs `tsc && vite build` — `tsc` with `noEmit: true` won't emit anything and will fail when it encounters `.tsx` extension imports because `allowImportingTsExtensions` requires `noEmit` to be `true` which it is, but `moduleResolution: "bundler"` is only available in TypeScript 5.0+. That's fine here.

The actual problem: `"moduleResolution": "bundler"` is incompatible with `"module": "ESNext"` when running `tsc` for **type-check in strict mode in TS 5.4** — specifically, Vite projects should use a separate `tsconfig.node.json` for `vite.config.ts` (which uses `__dirname` — a Node.js global not available in `"moduleResolution": "bundler"` context).

The standard Vite + React + TS setup uses two `tsconfig` files: one for the app (browser) and one for `vite.config.ts` (Node). Let me fix this properly:

Edited 2 files: 16s

`tsconfig.node.json` needs `@types/node` so `__dirname` resolves. Add it to `devDependencies`:

Applied diff to `frontend/package.json`

Three changes were made:

- `tsconfig.json` — Added `useDefineForClassFields`, `skipLibCheck`, `noFallthroughCasesInSwitch`; removed `"tests"` from `include` (test files use their own config); added `"references"` pointing to `tsconfig.node.json` — the standard Vite scaffold pattern
- `tsconfig.node.json` (new) — Separate config for `vite.config.ts` with `composite: true`, `"types": ["node"]` so `__dirname` and `path` resolve correctly in the Vite config file
- `package.json` — Added `@types/node` to `devDependencies` so `path.resolve` and `__dirname` in `vite.config.ts` don't produce type errors

The root cause was that `vite.config.ts` uses Node.js globals (`__dirname`, `path`) but the single `tsconfig.json` was configured for browser/bundler mode only — `moduleResolution: "bundler"` doesn't include Node types, causing TypeScript to error on them. The solution is the standard two-`tsconfig` split that the Vite scaffolder (`npm create vite`) generates by default.

76 files changed

Undo all Show all

Follow up or start new task Ctrl+N